

Intelligent Sudoku Solver Using CSP, AC-3, Backtracking and MRV

Student ID: 0112330669

Course: CSE 3812 Artificial Intelligence Laboratory

Summer 2026

Contents

1	Introduction	3
2	Problem Representation as CSP	3
2.1	Variables	3
2.2	Domains	3
2.3	Constraints	3
3	Input and Output Specification	4
4	Algorithm Design	4
4.1	AC-3 Algorithm	4
4.2	Backtracking Search	4
4.3	Minimum Remaining Values (MRV) Heuristic	5
5	Object-Oriented Implementation	5
6	Experimental Results and Ablation Study	5
6.1	MRV Enabled Results	6
6.2	MRV Disabled Results	6
7	Discussion	6
8	Conclusion	6
9	Future Improvements	7

1 Introduction

Sudoku is a classic constraint-based puzzle that can be modeled as a Constraint Satisfaction Problem (CSP). The objective of this project is to develop an Artificial Intelligence based Sudoku solver that can solve Sudoku puzzles efficiently.

Instead of using a naive brute-force approach, this project implements intelligent search and constraint propagation techniques. The implemented methods include CSP representation, AC-3 (Arc Consistency-3) algorithm, Backtracking Search, and Minimum Remaining Values (MRV) heuristic.

The main goal is to reduce unnecessary search operations and improve the efficiency of solving Sudoku puzzles.

2 Problem Representation as CSP

A Constraint Satisfaction Problem consists of three main components: variables, domains, and constraints.

2.1 Variables

The Sudoku board contains 81 variables where each individual cell represents one variable.

$$Total\ Variables = 9 \times 9 = 81$$

2.2 Domains

The domain represents the possible values that can be assigned to each variable.

For an empty cell:

$$Domain = \{1, 2, 3, 4, 5, 6, 7, 8, 9\}$$

For a pre-filled cell:

$$Domain = \{Given\ Value\}$$

2.3 Constraints

Sudoku contains three main constraints:

- No duplicate number can exist in the same row.
- No duplicate number can exist in the same column.
- No duplicate number can exist inside a 3x3 sub-grid.

3 Input and Output Specification

The program reads Sudoku puzzles from a text file named:

`puzzles.txt`

Each puzzle is represented as an 81-character string.

The character 0 represents an unassigned cell.

After solving, the program generates:

`solutions.txt`

The output file contains solved Sudoku boards represented as 81-character strings without any zero values.

4 Algorithm Design

4.1 AC-3 Algorithm

AC-3 (Arc Consistency-3) is used as a preprocessing algorithm.

The purpose of AC-3 is to reduce variable domains by removing values that cannot satisfy Sudoku constraints.

The algorithm maintains a queue of arcs:

$$(X_i, X_j)$$

For every arc, AC-3 checks whether each value in X_i has a valid supporting value in X_j .

If no supporting value exists, that value is removed from the domain.

Advantages of AC-3:

- Reduces search space.
- Removes inconsistent values before searching.
- Improves backtracking efficiency.

4.2 Backtracking Search

After applying AC-3, if the puzzle is not completely solved, recursive Backtracking Search is applied.

The process:

1. Select an unassigned variable.
2. Try a possible value.
3. Check constraint satisfaction.
4. Continue recursively.
5. Backtrack if contradiction occurs.

4.3 Minimum Remaining Values (MRV) Heuristic

MRV is a variable selection heuristic.

Instead of selecting variables randomly, MRV selects the variable with the smallest remaining domain.

Example:

$$\text{Cell } A = \{1, 2, 3, 4\}$$

$$\text{Cell } B = \{7\}$$

MRV selects Cell B first.

Benefits:

- Reduces unnecessary branching.
- Detects failure earlier.
- Decreases number of backtracks.

5 Object-Oriented Implementation

The solver is implemented using an object-oriented approach.

The main class is:

SudokuCSP

The class manages:

- Variables and domains
- Neighbor generation
- Constraint checking
- AC-3 execution
- Backtracking search
- Solution generation

This structure improves code organization, readability, and maintainability.

6 Experimental Results and Ablation Study

To evaluate the effect of the MRV heuristic, two experiments were performed:

1. MRV Enabled
2. MRV Disabled

Puzzle	Execution Time	Node Expansion
Puzzle 1	0.00548 sec	1
Puzzle 2	8.4245 sec	6337
Puzzle 3	1.3556 sec	950

Table 1: Performance with MRV Enabled

Puzzle	Execution Time	Node Expansion
Puzzle 1	0.00640 sec	1
Puzzle 2	553.1327 sec	347581
Puzzle 3	0.2813 sec	177

Table 2: Performance with MRV Disabled

6.1 MRV Enabled Results

6.2 MRV Disabled Results

7 Discussion

The experimental results show that MRV significantly improves solver performance for difficult Sudoku puzzles.

For Puzzle 2:

Method	Time	Nodes
MRV Enabled	8.4245 sec	6337
MRV Disabled	553.1327 sec	347581

Without MRV, the solver explores many unnecessary branches because variables are selected sequentially.

With MRV, the solver chooses the most constrained variable first, reducing the search space and detecting incorrect paths earlier.

8 Conclusion

An intelligent Sudoku solver was successfully developed using Artificial Intelligence techniques.

The Sudoku puzzle was modeled as a Constraint Satisfaction Problem and solved using:

- CSP representation
- AC-3 constraint propagation
- Backtracking Search
- MRV heuristic

The experimental results demonstrate that heuristic-based search provides better performance compared to sequential variable selection.

9 Future Improvements

Possible future improvements include:

- Degree Heuristic implementation.
- Least Constraining Value ordering.
- Visualization of solving process.
- Support for larger Sudoku variants.

References

1. Stuart Russell and Peter Norvig, Artificial Intelligence: A Modern Approach.
2. Artificial Intelligence Laboratory Course Materials.